

Processes

Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF
GOTHENBURG

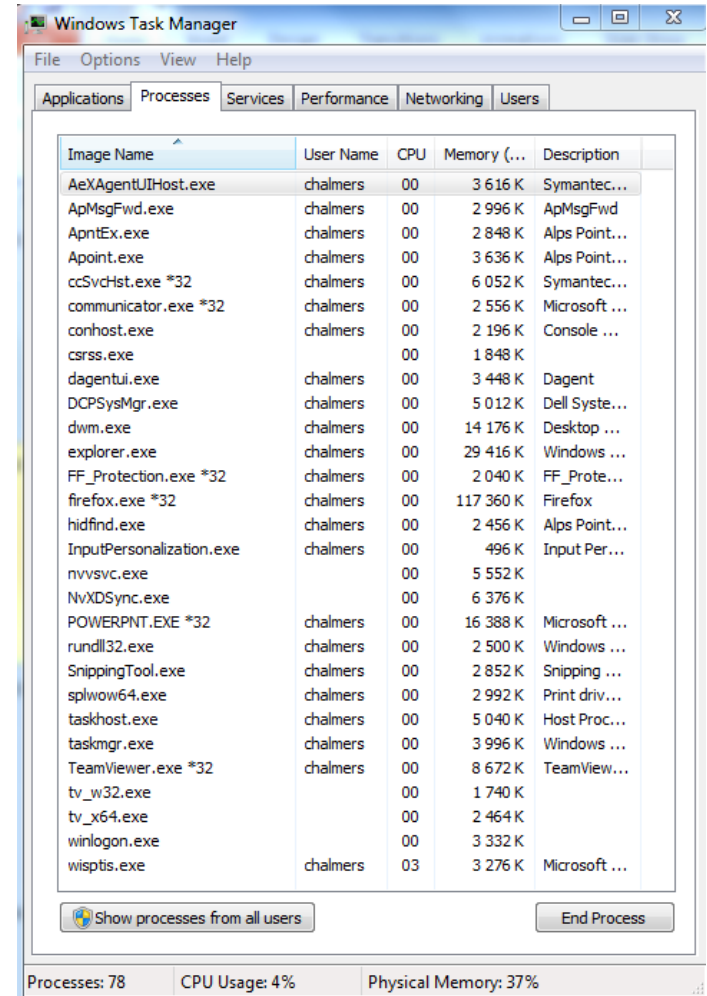
What to read (Main textbook)

- Chapter 2.1

(optional additional reading: 3.1-3.4, 3.5.3, 3.6.3 from Silberschatz
Operating System Concepts)

Objectives

- To introduce the notion of a process: a program in execution, which forms the basis of all computation
- To describe the various features of processes, including scheduling, creation and termination, and communication
- To explore interprocess communication using shared memory and message passing



AGENDA

- Processes (Introduction)
- Process Scheduling
- Operations on Processes
- Interprocess Communication (contains self-reading part)

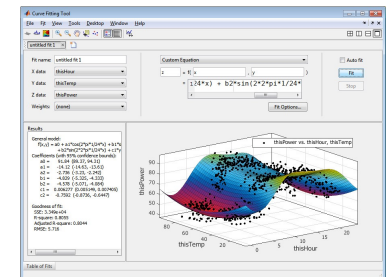
AGENDA

- Processes (Introduction)
- Process Scheduling
- Operations on Processes
- Interprocess Communication (contains self-reading part)

1. We run several programs at the same time

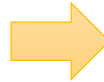


2. One CPU can only run one program at a time



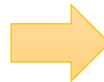
Concurrent vs Parallel execution

~~1. We run several programs at the same time~~



1. We feel several programs run at the same time

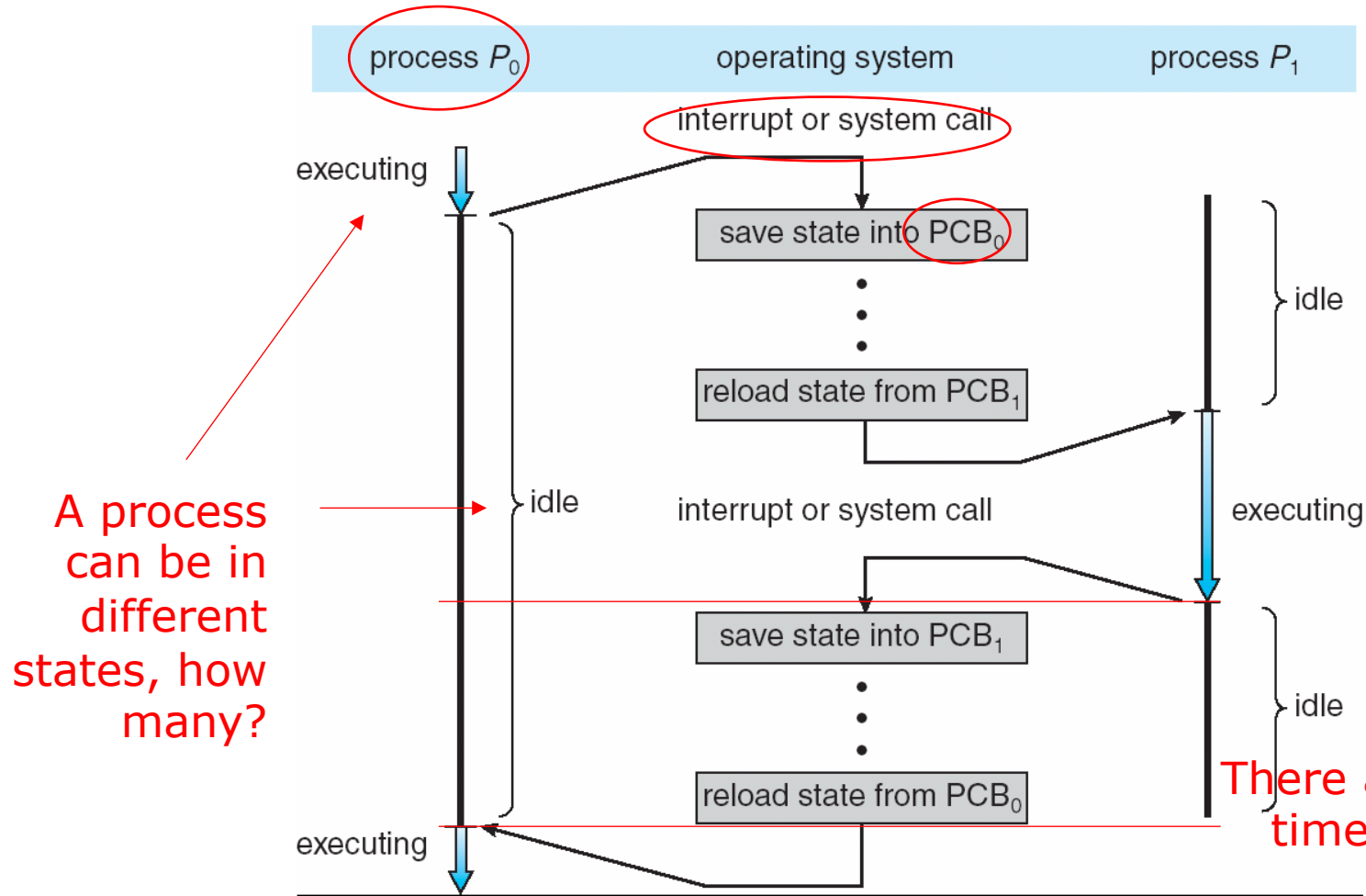
~~2. One CPU can only run one program at a time~~



2. Each CPU core can only run one program at the time

(Next lecture)

Process, not program

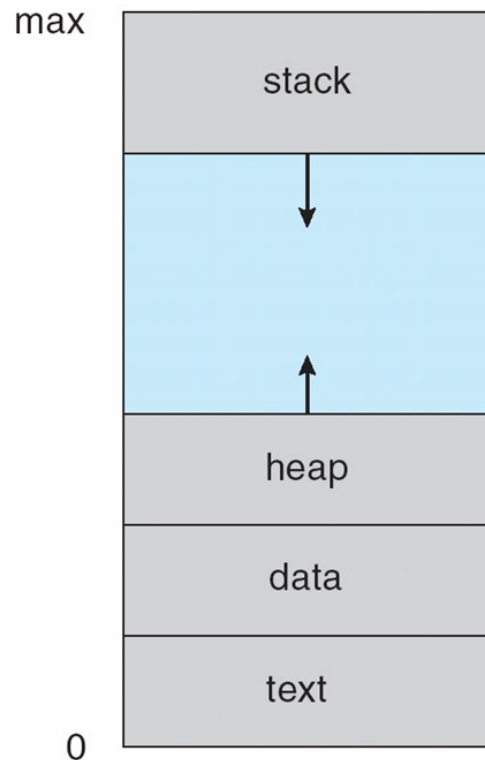


There are periods of time during which no process is running! Overheads should be minimal

Process Concept

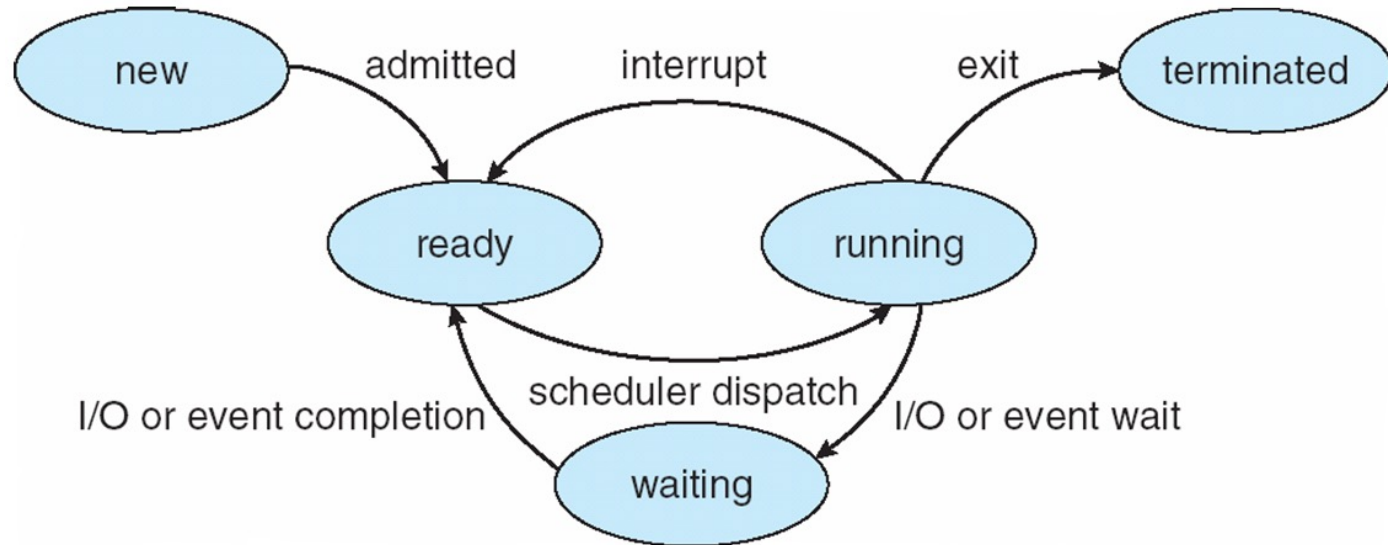
- Process (or job, task) – a program in execution; process execution must progress in sequential fashion
- A program is passive entity stored on disk (executable file), process is active
- A program becomes process when executable file loaded into memory
- The execution of a program is started via GUI mouse clicks, command line entry of its name, ...
- One program can have several process instances (consider multiple users executing the same program)
- A process can be the execution environment for other code (e.g., Java Virtual Machine)

Process in Memory



- Multiple parts:
 - The program code, also called text section
 - Data section containing global variables
 - Heap containing memory dynamically allocated during run time
 - Stack containing temporary data
 - Function parameters, return addresses, local variables

Diagram of Process State



- As a process executes, it changes state
 - **new**: The process is being created
 - **ready**: The process is waiting to be assigned to a processor
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **terminated**: The process has finished execution

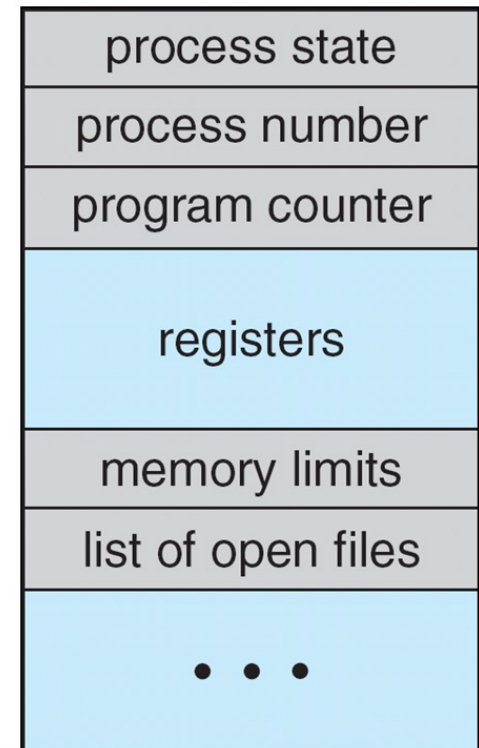
Important: only 1 process
can be **running** on any
CPU core / logical
processor at any instant.

Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch
- Context of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS and the PCB → the longer the context-switch time
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

Process Control Block (PCB)

- Information associated with each process in the OS
- Process state: running, waiting, etc.
- Program counter: location of next instruction to execute
- CPU registers: contents of all process-centric registers
- CPU scheduling information: priorities, scheduling queue pointers
- Memory-management information: memory allocated to the process
- Accounting information: CPU time used, clock time elapsed since start, time limits
- I/O status information: I/O devices allocated to the process, list of open files



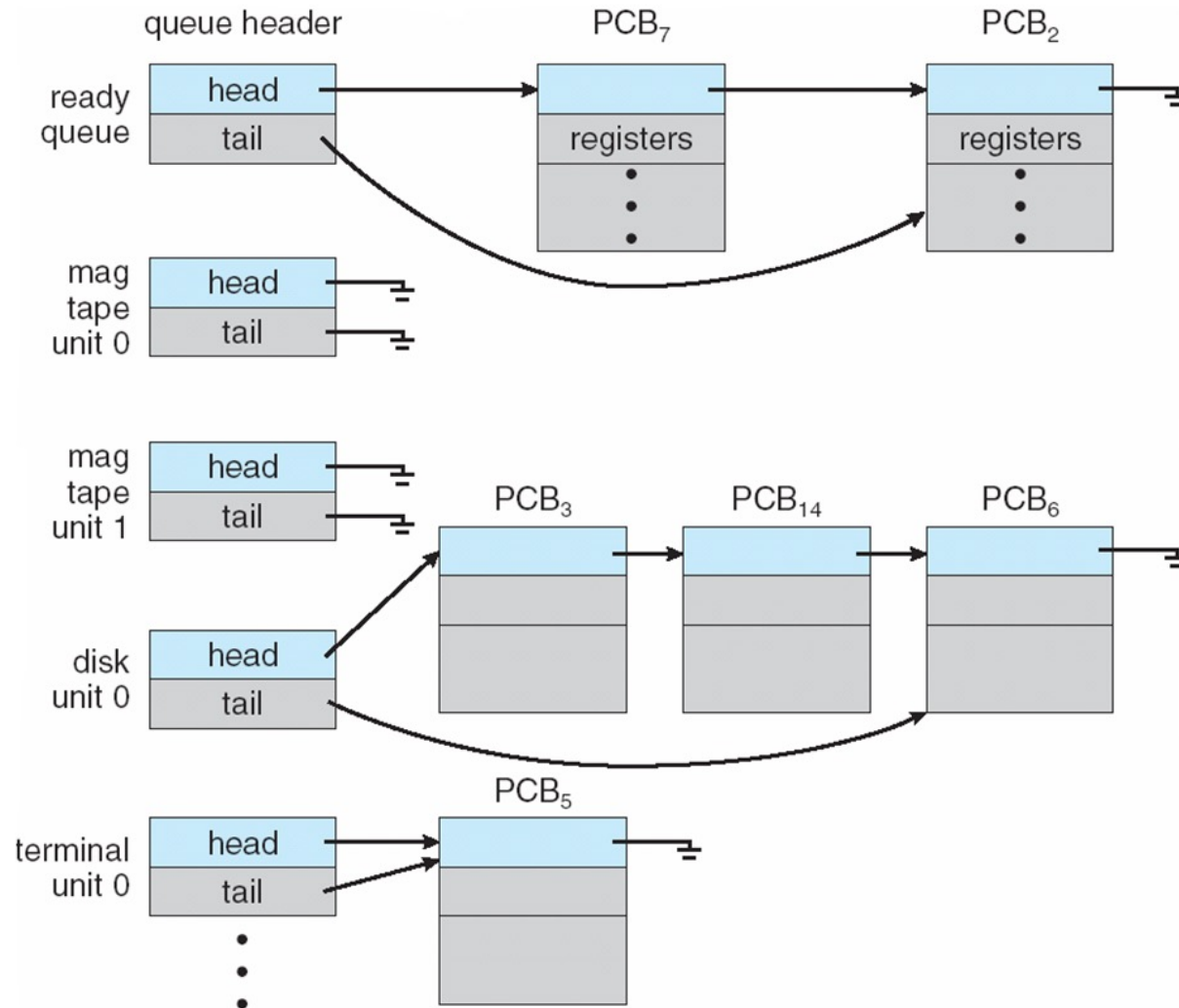
AGENDA

- Processes (Introduction)
- Process Scheduling
- Operations on Processes
- Interprocess Communication (contains self-reading part)

Process Scheduling

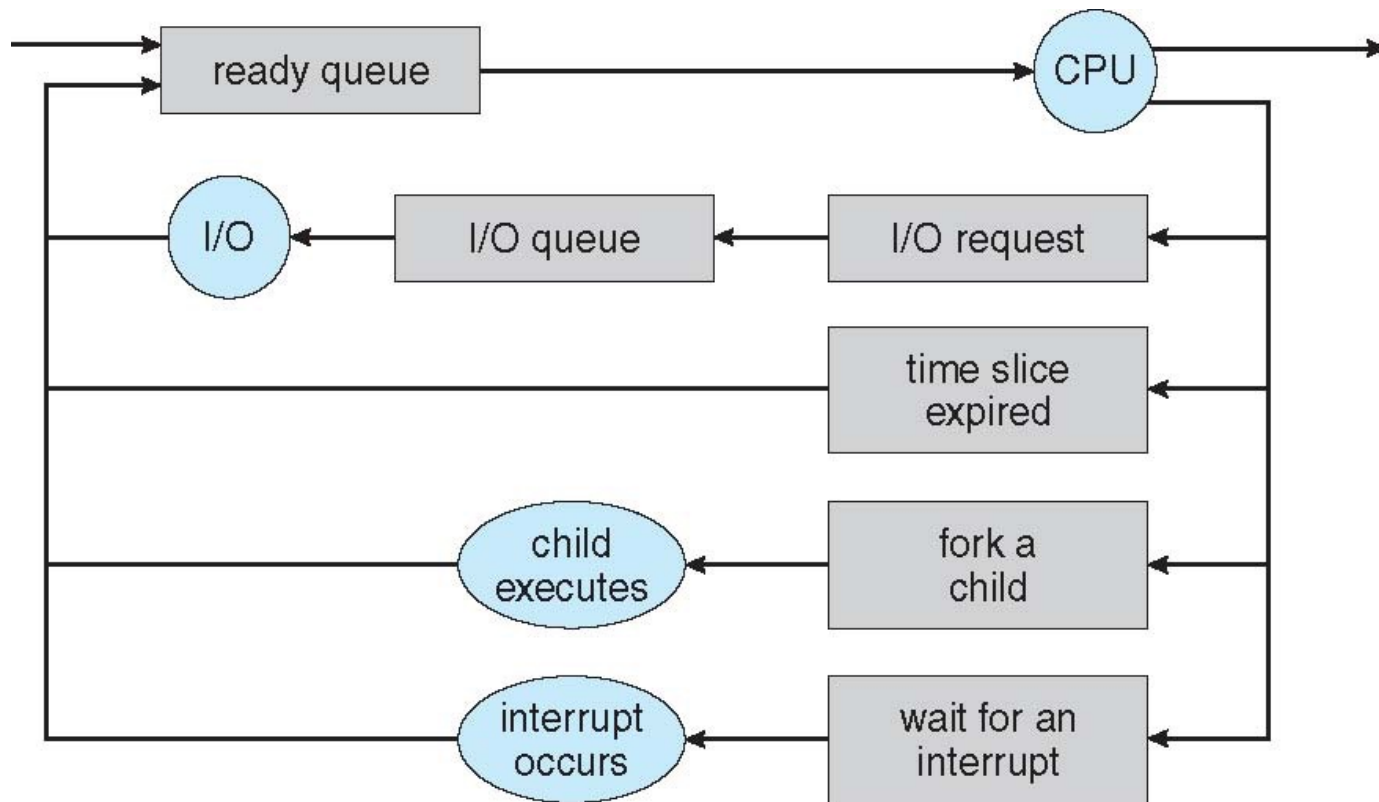
- Maximize CPU use, quickly switch processes onto CPU for time sharing
- Process scheduler selects among available processes for the next execution on CPU
- Maintains scheduling queues of processes
 - Job queue – set of all processes in the system
 - Ready queue – set of all processes residing in main memory, ready and waiting to execute
 - Device queues – set of processes waiting for an I/O device
 - Processes migrate among the various queues

Ready Queue And Various I/O Device Queues



Representation of Process Scheduling

Queueing diagram represents queues, resources, flows



Schedulers

- Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates the CPU
 - Sometimes the only scheduler in a system
 - Short-term scheduler is invoked frequently (milliseconds) $\Rightarrow$ (must be fast)
- Long-term scheduler (or job scheduler) – selects which processes should be brought into the ready queue
 - Long-term scheduler is invoked infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
 - The long-term scheduler controls the degree of multiprogramming

AGENDA

- Processes (Introduction)
- Process Scheduling
- **Operations on Processes**
- Interprocess Communication (contains self-reading part)

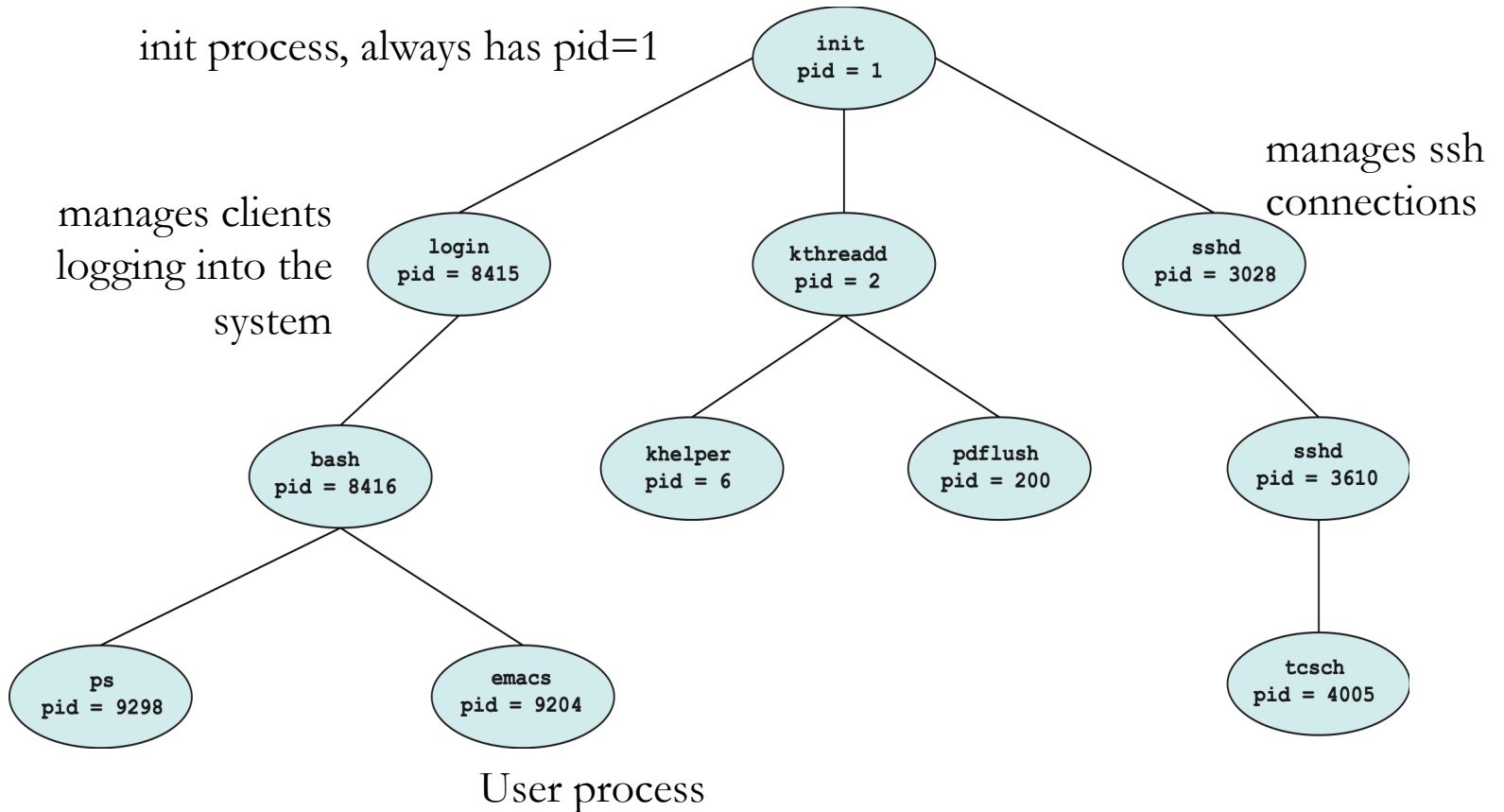
Operations on Processes

- The system must provide mechanisms for:
 - process creation, and
 - process termination (as detailed in the following slides).

Process Creation

- A parent process creates children processes, which, in turn, create other processes, forming a tree of processes
- Generally, a process is identified and managed via a process identifier (pid)
- Resource sharing options
 - Parent and children share all resources
 - Children share a subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

A Tree of Processes in Linux



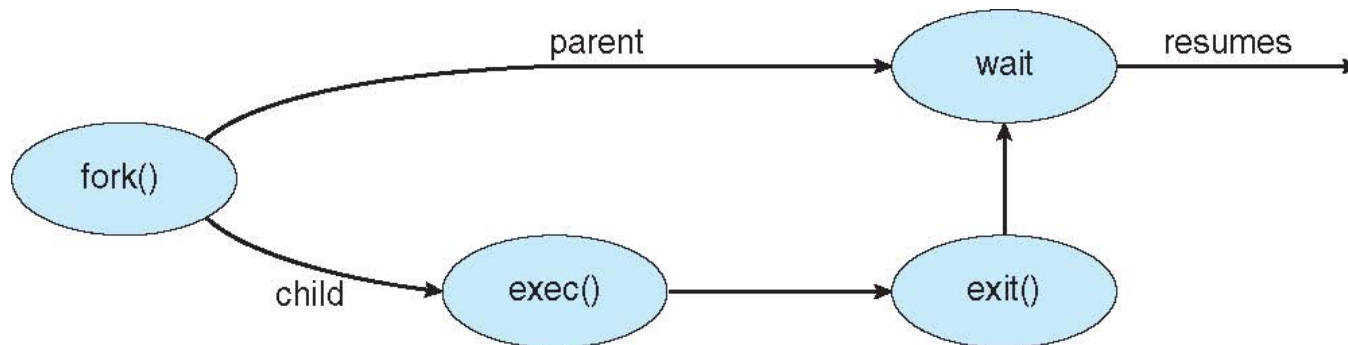
pstree -pl

```
XXXXXXXXXX:~$ pstree -pl
```

```
init(1)-+-NetworkManager(1215)-+-dhclient(3386)
      |                               |-dnsmasq(3390)
      |                               |-{NetworkManager}(1224)
      |                               `--{NetworkManager}(2164)
      |-accounts-daemon(1527)---{accounts-daemon}(1530)
      |-acpid(1360)
      |-apache2(16861)-+-apache2(2458)
      |                   |-apache2(2460)
      |                   |-apache2(2461)
      |                   |-apache2(2462)
      |                   |-apache2(2463)
      |                   |-apache2(2464)
      |                   |-apache2(4079)
      |                   |-apache2(4082)
      |                   |-apache2(5464)
      |                   |-apache2(5465)
      |                   `--apache2(5466)
      |-at-spi-bus-laun(3570)-+-dbus-daemon(3576)
      ...
```

Process Creation (Cont.)

- Address space
 - The child is a duplicate of parent
 - The child has a program loaded into it
- UNIX examples
 - `fork()` system call creates new process
 - `exec()` system call used after a `fork()` to replace the process' memory space with a new program



C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
    pid_t pid;
```

```
    /* fork a child process */
    pid = fork();
```

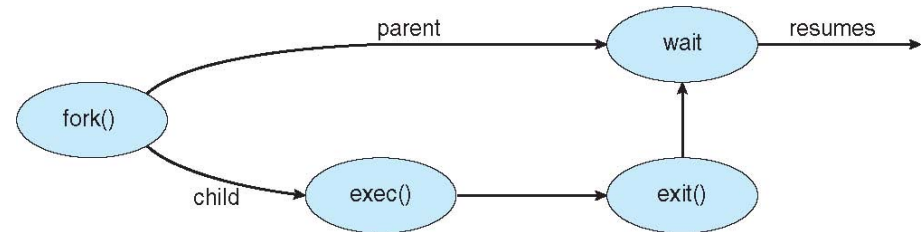
```
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
```

```
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
```

```
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }
```

```
    return 0;
```

```
}
```



Process Termination

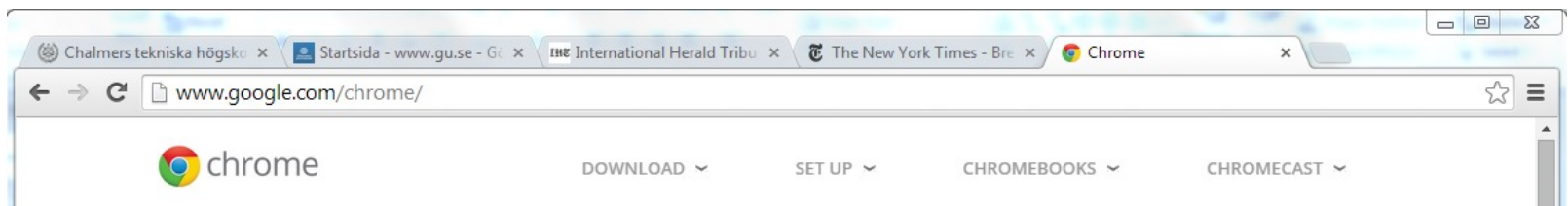
- Process executes last statement and then asks the operating system to delete it using the `exit()` system call.
 - The system call allows to return status data from the child to the parent (if parent used `wait()`)
 - The process' resources are deallocated by the operating system
- Parent may terminate the execution of children process using the `kill()` system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow a child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - Cascading termination. All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the `wait()` system call. The call returns status information and the pid of the terminated process
- `pid = wait(&status);`
- A terminated child becomes a zombie when its status has not yet been collected by its parent
- If the parent has terminated without invoking `wait`, the process is an orphan

Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
 - If one website causes trouble, the entire browser can hang or crash
- Google Chrome Browser is multiprocess, with 3 different types of processes:
 - Browser process manages user interface, disk and network I/O
 - Renderer process renders web pages, deals with HTML, JavaScript. A new renderer is created for each website opened
 - Runs in a sandbox to restrict disk and network I/O and minimize risks of security exploits
 - Plug-in process for each type of plug-in



AGENDA

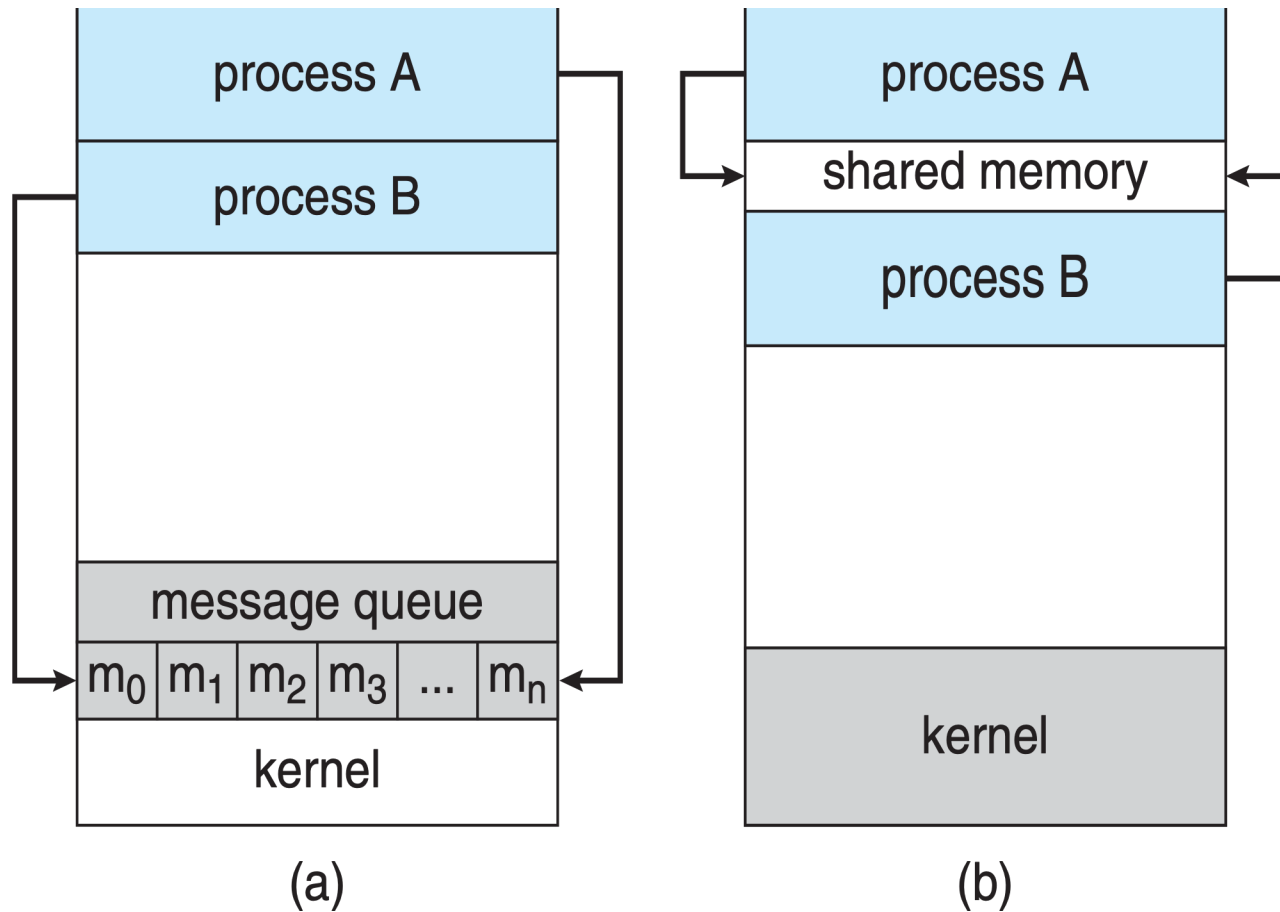
- Processes (Introduction)
- Process Scheduling
- Operations on Processes
- Interprocess Communication (contains self-reading part)

Interprocess Communication

- Processes within a system may be independent or cooperating
- Cooperating processes can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
- Cooperating processes need interprocess communication (IPC)
- Two models of IPC
 - Shared memory
 - Message passing

Communication Models

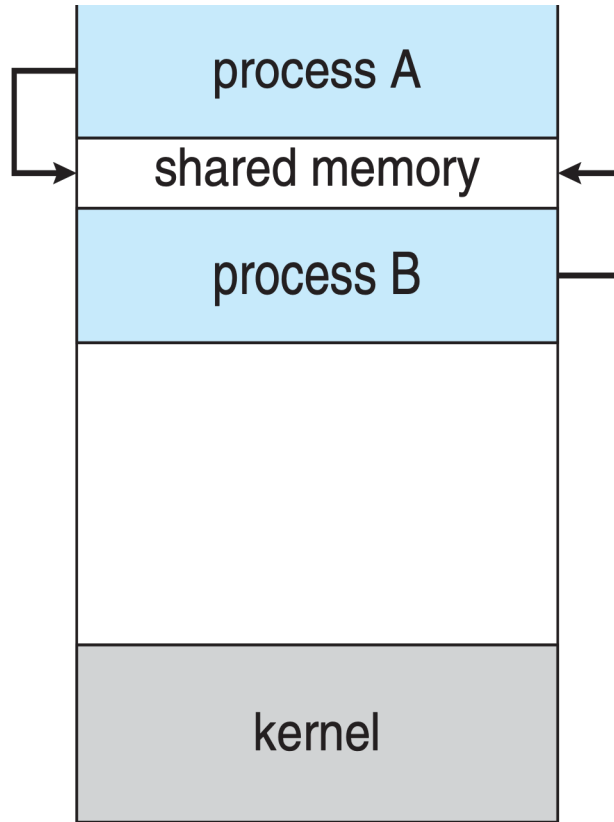
(a) Message passing. (b) Shared memory.



Cooperating processes: Producer-Consumer Problem

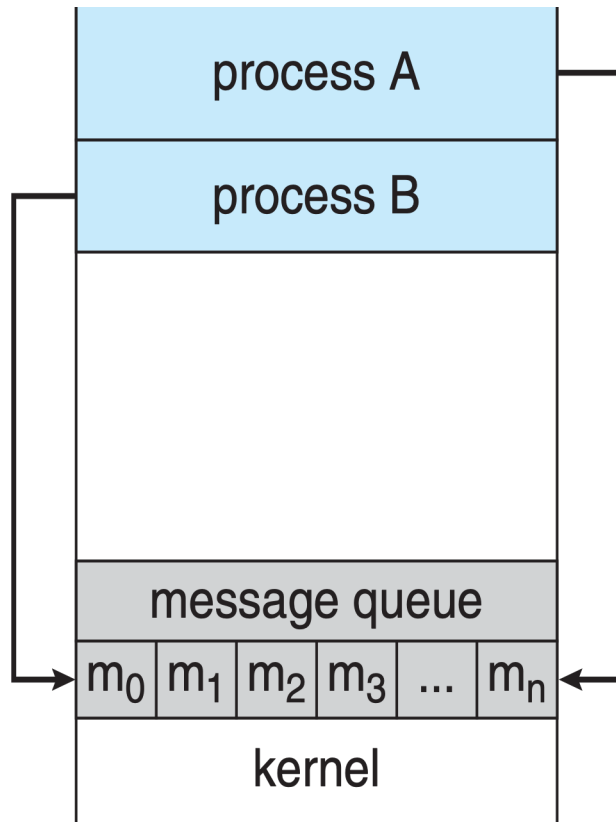
- Paradigm for cooperating processes: a producer process produces information that is consumed by a consumer process
- **Unbounded-buffer:** places no practical limit on the size of the buffer
- **Bounded-buffer:** assumes that there is a fixed buffer size

Interprocess Communication – Shared Memory



- The major issue is to provide a mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in detail in the following lessons

Interprocess Communication – Message Passing



- If processes P and Q wish to communicate, they need to:
 - Establish a communication link between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Is the size of a message that the link can accommodate fixed or variable?
 - Is a link unidirectional or bidirectional?

Logical implementation – Issues [Self-reading section]

- Naming, Direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

Logical implementation – Issues

- Naming: direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

Direct Communication

- Processes must name each other explicitly:
 - `send (P, message)` – send a message to process P
 - `receive(Q, message)` – receive a message from process Q
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bidirectional

Indirect Communication

- Messages are sent to and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique ID
 - Processes can communicate only if they share a mailbox
- Properties of a communication link
 - A link is established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - A link may be unidirectional or bidirectional

Indirect Communication

- Operations
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
 - `send(A, message)` – send a message to mailbox A
 - `receive(A, message)` – receive a message from mailbox A

Logical implementation – Issues

- Naming: direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

Synchronization

- Message passing may be either blocking or non-blocking
- Blocking is considered synchronous
 - Blocking send - the sender is blocked until the message is received
 - Blocking receive - the receiver is blocked until a message is available
- Non-blocking is considered asynchronous
 - Non-blocking send - the sender sends the message and continues
 - Non-blocking receive - the receiver receives:
 - A valid message, or
 - A Null message
- Different combinations possible
 - If both send and receive are blocking, we have a rendezvous

Synchronization (Cont.)

- Producer-consumer becomes trivial for blocking send/receive:

```
message next_produced;  
while (true) {  
    /* produce an item in next produced */  
    send(next_produced);  
}
```

```
message next_consumed;  
while (true) {  
    receive(next_consumed);  
  
    /* consume the item in next consumed */  
}
```

Logical implementation – Issues

- Naming: direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

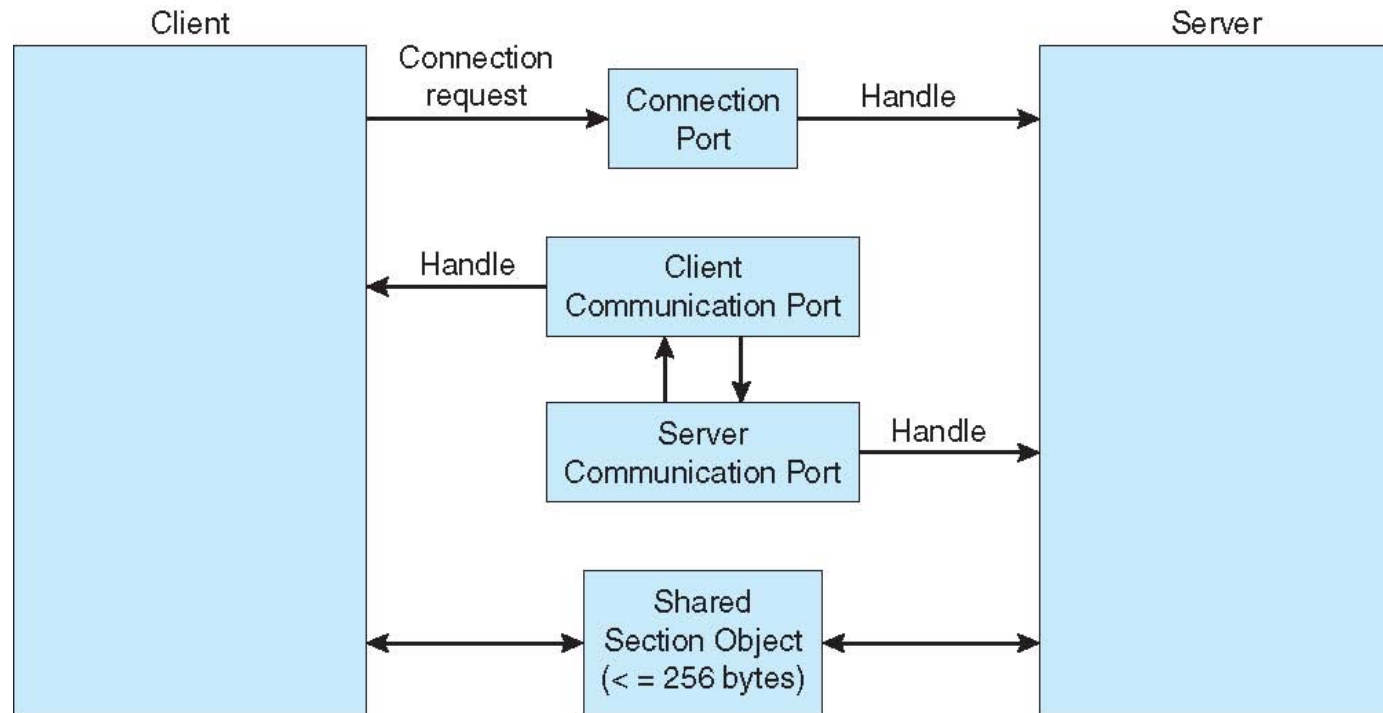
Buffering

- A queue of messages is attached to the link, and is implemented in one of three ways:
 1. Zero capacity – no messages are queued on a link: the sender must wait for receiver (rendezvous)
 2. Bounded capacity – finite length of n messages: the sender must wait if the link is full
 3. Unbounded capacity – infinite length: the sender never waits

Examples of IPC Systems – Windows

- Message-passing centric via advanced local procedure call (ALPC) facility
- Only works between processes on the same system
- Uses ports (like mailboxes) to establish and maintain communication channels

Local Procedure Calls in Windows

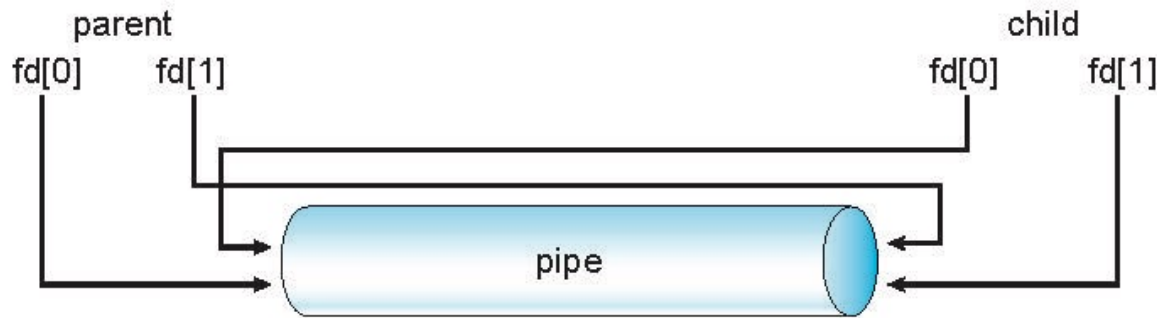


Pipes

- Pipes act as a conduit allowing two processes to communicate
- Ordinary pipes – usually used by a parent process to communicate with a child process that it created.
 - They can also be shared with grandchildren or other related processes that inherit those descriptors. On UNIX, a process may also pass a pipe file descriptor to another process through a UNIX-domain socket.
- Named pipes – can be accessed in cases not covered by ordinary pipes (by name).

Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



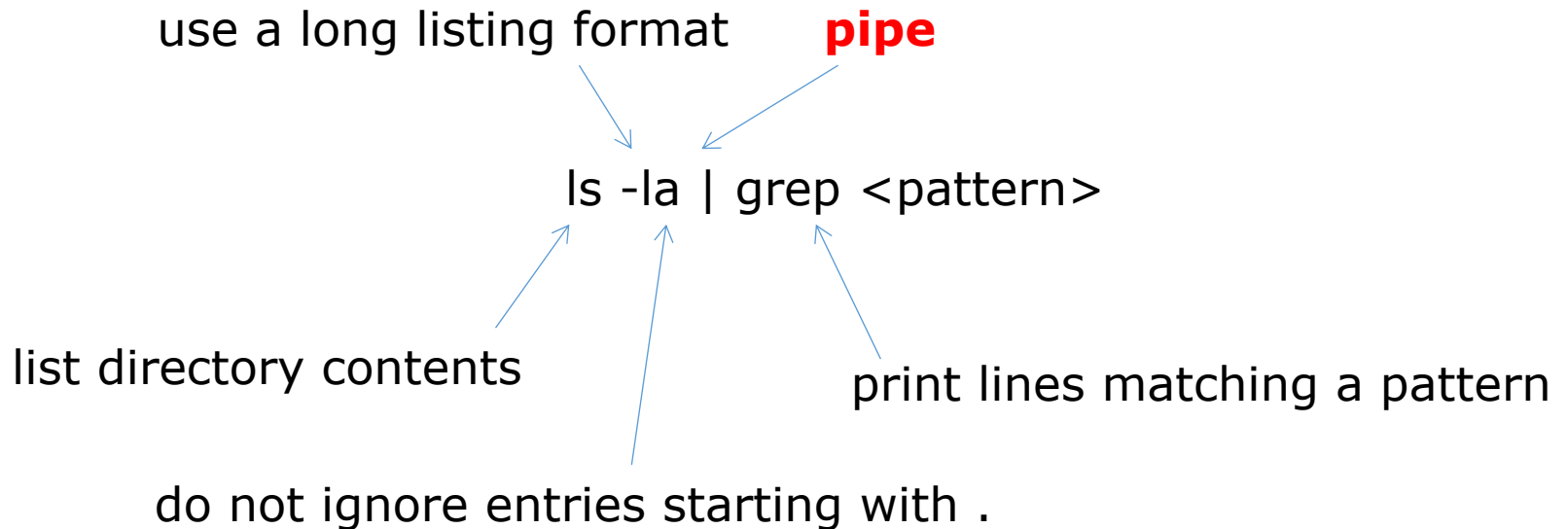
- Windows calls these **anonymous pipes**
- See Unix and Windows code samples in textbook

Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Named pipes are provided on both UNIX and Windows systems

Pipes in UNIX

- Send the output of one command to the input of another command



Pipes in UNIX

ls -la

```
vincenzo [redacted]:~$ ls -la
total 132
drwxr-xr-x 13 vincenzo vincenzo 4096 Nov  1 15:04
drwxr-xr-x 14 root      root      4096 Oct 28 2013
-rw----- 1 vincenzo vincenzo 52882 Oct 31 22:27
-rw-r--r-- 1 vincenzo vincenzo 220 Aug  7 2013
-rw-r--r-- 1 vincenzo vincenzo 3785 Jun 23 10:10
drwx----- 2 vincenzo vincenzo 4096 Feb 18 2014
drwx----- 3 vincenzo vincenzo 4096 Jul 18 13:10
drwxrwxr-x 5 vincenzo vincenzo 4096 Jul 16 08:51
drwxrwxr-x 6 vincenzo vincenzo 4096 Sep  7 19:12
drwxrwxr-x 3 vincenzo vincenzo 4096 Feb 20 2014
drwxrwxr-x 12 vincenzo vincenzo 4096 Oct 24 15:11
-rw----- 1 vincenzo vincenzo 35 Nov  1 15:04
drwxrwxr-x 17 vincenzo vincenzo 4096 Jul 21 10:30
drwxrwxr-x 3 vincenzo vincenzo 4096 Mar 26 2014
drwxrwxr-x 14 vincenzo vincenzo 4096 Apr 27 2014
-rw-r--r-- 1 vincenzo vincenzo 675 Aug  7 2013
drwx----- 2 vincenzo vincenzo 4096 Feb 20 2014
drwxrwxr-x 3 vincenzo vincenzo 4096 Jul 15 09:02
-rw----- 1 root      root      737 Apr  8 2014
-rw----- 1 vincenzo vincenzo 51 Jun 25 08:58
```

ls -la | grep Apr

```
vincenzo [redacted]:~$ ls -la | grep Apr
drwxrwxr-x 14 vincenzo vincenzo 4096 Apr 27 2014
-rw----- 1 root      root      737 Apr  8 2014
```

Ordinary Pipes - example

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>

#define BUFFER_SIZE 25
#define READ_END    0
#define WRITE_END   1
```

Ordinary Pipes - example

```
1. int main(void)
2. {
3.     char write_msg[BUFFER_SIZE] = "Greetings";
4.     char read_msg[BUFFER_SIZE];
5.     pid_t pid;
6.     int fd[2];

7.     /* create the pipe */
8.     if (pipe(fd) == -1) {
9.         fprintf(stderr, "Pipe failed");
10.        return 1;
11.    }

12.    /* now fork a child process */
13.    pid = fork();
```

```

14.if (pid < 0) {
15.    fprintf(stderr, "Fork failed");
16.    return 1;
17.}

18.    if (pid > 0) { /* parent process */
19.        /* close the unused end of the pipe */
20.        close(fd[READ_END]);
21.        /* write to the pipe */
22.        write(fd[WRITE_END], write_msg,
    strlen(write_msg)+1);
23.        /* close the write end of the pipe */
24.        close(fd[WRITE_END]);
25.    }
26.    else { /* child process */
27.        /* close the unused end of the pipe */
28.        close(fd[WRITE_END]);
29.        /* read from the pipe */
30.        read(fd[READ_END], read_msg, BUFFER_SIZE);
31.        printf("child read %s\n", read_msg);
32.        /* close the read end of the pipe */
33.        close(fd[READ_END]);
34.    }
35.    return 0;
36.}

```

Questions

The instruction to be run next when process X is scheduled is maintained in ...

1. its text section
2. its data section
3. its process control block

Which of the following state transitions is not valid?

1. ready --> running
2. running --> ready
3. running --> waiting
4. ready --> waiting

During a context switch, what is saved by the OS (to be later re-loaded) is ...

1. the CPU context into the PCB of the process
2. the CPU context into the PCB of the process + the memory of the process
3. the memory of the process

Which of the following statements is false?

1. Line 4 can be executed by the parent AND the child process
2. Line 7 is executed by the parent process
3. Line 7 is executed by the child process

```
1 pid = fork();
2
3 if (pid < 0) {
4     [instruction 1]
5 }
6 else if (pid == 0) {
7     [instruction 2]
8 }
9 else {
10    [instruction 3]
11 }
```

Which type of pipes require a parent-child relationship between two processes?

1. Ordinary
2. Named
3. None of the above

Including the initial parent process, how many processes are created by this code?

1. 4
2. 5
3. 16
4. 31

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     int i;
7     for (i=0; i<4; i++)
8         fork();
9
10    return 0;
11 }
```

Question 2

What is printed by this program?

```
1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <unistd.h>
4
5  #define SIZE 5
6
7  int nums[SIZE] = {0,1,2,3,4};
8
9  int main()
10 {
11     int i;
12     pid_t pid;
13     pid = fork();
14     if (pid == 0) {
15         for (i = 0; i < SIZE; i++) {
16             nums[i] *= -i;
17             printf("CHILD %d\n", nums[i]);
18         }
19     }
20     else if (pid > 0) {
21         wait(NULL);
22         for (i = 0; i < SIZE; i++)
23             printf("PARENT: %d\n", nums[i]);
24     }
25     return 0;
26 }
```

Question 3

What is printed by this program?

```
1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <unistd.h>
4
5  int main()
6  {
7      pid_t pid, pid1;
8      pid = fork();
9      if (pid < 0) {
10         fprintf(stderr, "Fork Failed");
11     }
12     else if (pid == 0) {
13         pid1 = getpid();
14         printf("child: pid = %d", pid)
15         printf("child: pid1 = %d", pid1)
16     }
17     else if (pid > 0) {
18         pid1 = getpid();
19         printf("parent: pid = %d", pid)
20         printf("parent: pid1 = %d", pid1)
21         wait(NULL);
22     }
23     return 0;
24 }
```

Question 4

Consider a multiprogrammed system with a degree of multiprogramming of 5 (i.e., five processes are present in memory and competing for CPU time). If each process spends 40% of its time waiting for I/O (independently of others), what will the CPU utilization be?